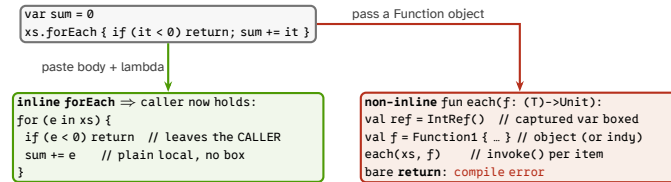


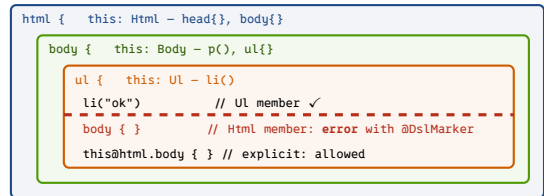
In one line: Kotlin's “syntax” is mostly **conventions the compiler rewrites**: a lambda typed `T.() -> R` runs with `this: T` (builders, Gradle, Compose); `inline` pastes a function *and its lambdas* into the caller (no allocation, return leaves the caller, reified works); operator/infix map symbols to named functions; `by` forwards a property or a whole interface to another object. None of it is dynamic — all resolved at compile time.

One call, two compilations



`noinline f`: keep this lambda an object (store / pass on) · `crossinline f`: inlined but called from another context (Runnable, object) ⇒ `return@forEach` = skip one item

Receiver scopes in a DSL



`@DslMarker` on a common annotation: only the **innermost** marked receiver is implicit — outer ones need `this@label`. Without it, `body {}` inside `ul {}` compiles.

Inline — mechanism & rules

- Wins: no Function object per call, no Ref boxes for captured vars, non-local return, reified T; break/ continue inside inline lambdas stable since 2.2. Cost: bytecode grows per call site; no lambda param ⇒ IDE warns “insignificant”.
- A public `inline` body is copied into *other* modules, so it may only touch public API — or `@PublishedApi` internal. Changing it needs callers recompiled.
- Without inline, a lambda compiles to `invokedynamic` (2.0 default) or a class; `return@label` is the only return allowed in it.

Operator conventions & infix

you write	compiler calls	you write	compiler calls
<code>a + b</code>	<code>a.plus(b)</code>	<code>a[i] = v</code>	<code>a.set(i, v)</code>
<code>a += b</code>	<code>plusAssign, else a = a + b</code>	<code>f(x)</code>	<code>f.invoke(x)</code>
<code>x in c</code>	<code>c.contains(x)</code>	<code>a..b, a.<b</code>	<code>rangeTo, rangeUntil</code>
<code>a < b</code>	<code>a.compareTo(b) < 0</code>	<code>val (x, y) = p</code>	<code>component1(), 2()</code>
<code>a == b</code>	<code>a?.equals(b) ?: (b === null)</code>	<code>+ "txt", -a</code>	<code>unaryPlus, unaryMinus</code>

Each needs the operator modifier (member or extension).
`infix fun A.to(b: B) ⇒ a to b` (one param, no default). `==` can only be customised via `equals`; `===` is identity.

Extensions are static

- Compiled to static `f(Receiver r, ...)`: chosen by the **declared** type. `val s: Shape = Rect(); s.name()` calls `Shape.name()`.
- A **member always wins** over an extension with the same signature (warning: “shadowed”); extensions can't reach private members.
- Nullable receivers (`fun String?.orEmpty()`), companion extensions (`fun Foo.Companion.of()`); extension *properties* have no backing field — computed only. Declared inside a class = scoped to it (two receivers).

Example — a type-safe builder

```

@DslMarker annotation class HtmlDsl
@HtmlDsl abstract class Tag { val kids = mutableListOf<Tag>()
    protected fun <T> Tag add(t: T, init: T.() -> Unit) =
        t.apply(init).also { kids += it } }
class Html : Tag() { fun body(init: Body.() -> Unit) = add(Body(), init) }
class Body : Tag() { fun ul(init: Ul.() -> Unit) = add(Ul(), init) }
class Ul : Tag() { fun li(s: String) = add(Li(s)) {} }
fun html(init: Html.() -> Unit) = Html().apply(init) // entry point
val page = html { body { ul { li("a"); li("b") } } }

```

Property delegation

- `val x by d` compiles to a hidden `x$delegate` field; `get() = d.getValue(this, ::x)`, `set(v) = d.setValue(this, ::x, v)` — operator functions taking `(thisRef, property: KProperty<*>)`. Interfaces: `ReadOnlyProperty/ReadWriteProperty`.
- `by lazy` modes: `SYNCHRONIZED` (default, one lock, one init), `PUBLICATION` (may init twice, first wins), `NONE` (no sync — UI thread only). An exception in the initializer is rethrown; next access retries.
- `Delegates.observable(init) { p, old, new -> }` runs *after* the set; `vetoable` runs *before* and returns Boolean; `Delegates.notNull()` = lateinit for primitives. `by map` reads `map[property.name]`; `by this::other` aliases.
- `provideDelegate(thisRef, prop)`: runs once at *declaration* — validate or register the name before the first access.

Class delegation & context parameters

- `class Cached(r: Repo) : Repo by r` — generated forwarding methods; your overrides win, but `r`'s own internal calls never see them (forwarding, not inheritance).
- Context parameters**: preview 2.2 (–Xcontext-parameters), **stable 2.4.0** (Jul 2026), replacing context receivers. Used by name, passed *implicitly by type*; not on constructors.

```

context(log: Logger) fun sync(id: Int) { log.info("sync $id") }
context(_: Logger) fun syncAll() = ids.forEach { sync(it) } // passed on
fun main() = context(ConsoleLogger()) { syncAll() } // provide it
val token: String? by prefs.string("token") // custom delegate
val cfg by lazy(LazyThreadSafetyMode.NONE) { parse() } // main-thread only

```

Traps

- ! `return in forEach { }` ends the *function* — use `return@forEach`.
- ! “Override” an extension — impossible, declared type decides.
- ! `by lazy` holding a Context in a long-lived object = leak.

Remember: receiver lambdas build DSLs · inline pastes (return escapes) · operators are named functions · `by forwards`.